BCS2420 – Computer Security: Solution

2025-03-21

Below are the **model solutions** to the exam questions.

Part A

1. Which statement about the STRIDE threat-modeling method is correct?

   Answer: (b) STRIDE stands for Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Escalation of Privilege and helps ensure common threats are not overlooked.

   - **Why (b) is correct:** STRIDE is a mnemonic covering six major categories of threats, making sure analysts consider each possibility (Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Escalation of Privilege).

2. Risk equation (R = T ×V ×C). Which best describes (V) (vulnerability)?

   Answer: (c) The probability that the system, if attacked, will be successfully compromised.

   - **Why (c) is correct:** In the (R = T ×V ×C) model, (V) is how likely it is that a given threat attempt will succeed due to weaknesses in the system.

3. Which adversary attribute most directly addresses their technical means and skill set?

   Answer: (d) Capabilities.

   - **Why (d) is correct:** Capabilities generally refer to resources, knowledge, and skills the adversary possesses, including their computing power and technical proficiency.

4. Comparing symmetric vs. public-key (asymmetric) cryptography

   Answer: (c) Symmetric systems require a pre-shared secret for encryption/decryption, while public-key systems use separate public and private keys.

   - **Why (c) is correct:** Symmetric cryptography uses one shared key for both encryption and decryption, whereas public-key systems rely on two different keys (public and private).

5. Which statement about stream ciphers is true?

   Answer: (c) They encrypt data one bit/character at a time, often combined with a keystream generator.

   - **Why (c) is correct:** Stream ciphers process plaintext bit-by-bit (or byte-by-byte) in synchronization with a keystream.

6. In a typical digital signature workflow, why do we hash the message before signing?

   Answer: (b) To reduce the data length for faster signature operations and to detect any message alteration.

   - **Why (b) is correct:** Hashing produces a short digest. Signing this digest is more efficient than signing a large message directly, and any changes to the message change its hash.

7. The best reason to include a random unique salt for each user's stored password hash?

   Answer: (c) It prevents the use of precomputed hash tables (rainbow tables) across multiple users.

   - **Why (c) is correct:** A per-user salt forces attackers to compute hashes for each password attempt on a per-user basis rather than using global precomputed tables.

8. Which step slows offline password guessing after an attacker has the hashed file?

   Answer: (b) Using a specialized key-stretching algorithm with salts and high iteration counts.

   - **Why (b) is correct:** Key-stretching with many iterations (e.g., PBKDF2, bcrypt) slows down each guess, making large-scale offline attempts more expensive.

9. Fundamental difference between passive and active attackers

   Answer: (b) A passive attacker attempts to observe traffic, whereas an active attacker may inject or modify data.

- **Why (b) is correct:** Passive attackers stealthily eavesdrop. Active attackers can alter or insert messages into a channel.

10. "Default-allow" firewall policy

    Answer: (b) It allows traffic by default, creating a risk that unrecognized services remain accessible.

    - **Why (b) is correct:** A default-allow stance means unlisted traffic flows until a rule explicitly blocks it.

11. Malicious scripts posted by another user's comment, stored on the server

    Answer: (b) Stored cross-site scripting.

    - **Why (b) is correct:** The malicious payload is "persisted" on the server, then delivered to multiple visitors. This is the hallmark of stored XSS.

12. Which situation most clearly illustrates a man-in-the-middle attack?

    Answer: (b) An attacker intercepts key exchange messages and substitutes public keys, relaying data so both endpoints believe they communicate directly.

    - **Why (b) is correct:** This is the classic MITM scenario: the attacker sits in the middle of communication, altering or substituting keys.

Part B

B1. Formal security policy and threat modeling

- A formal security policy defines what actions are permitted or forbidden, clarifying system boundaries.
- Threat modeling uses this policy to identify possible violations that attackers might cause. A policy violation creates a **non-secure** state because it contradicts the organization's defined security rules.
- **Example:** If the policy states "No user may modify financial records except the finance group," then any threat vector allowing unauthorized modification breaks the policy, thus resulting in a non-secure state.

B2. Online vs. offline password guessing

- **Online** guessing: The attacker must interact with the legitimate server for each attempt, so rate-limiting or lockout policies can hinder guess volume.
- **Offline** guessing: The attacker has the hashed password file and can try unlimited guesses locally without server control.
  - **Defense for offline** attacks: Iterated hashing (e.g., PBKDF2, bcrypt).
  - **Why it's ineffective against online attacks:** If the attacker is brute-forcing online, the server can still enforce lockouts or rate-limits. The extra hashing cost on the server side helps somewhat but doesn't by itself block repeated attempts if the server is forced to respond. Real-time server policies are typically more effective for online scenarios.

B3. X.509 certificate binding and issuer signature

- **Key fields**:
  1. **Subject** (the entity name being certified),
  2. **Subject's Public Key** (and its algorithm details),
  3. **Issuer** (the Certification Authority),
  4. **Validity Period** (Not-Before and Not-After dates),
  5. **Digital Signature** by the CA.
- **Issuer signature**: Ensures the certificate is authentic and unmodified. Without the CA's trusted signature, anyone could create a fake certificate binding any public key to a legitimate name.

Part C

C1. Trojan horse vs. worm

- A **Trojan horse** masquerades as a legitimate program but contains hidden malicious code. Typically, it needs user action to download or run it.
- A **worm** spreads **automatically** by exploiting network or software vulnerabilities, not relying on the user to launch it explicitly.

C2. Kernel-mode rootkit discovery

1. **System call hooking**: The rootkit alters kernel tables or code so that legitimate queries (e.g., for process listings or file directories) are intercepted and modified, hiding malicious processes/files.
2. **Two steps to remove and reduce reinfection**:
    - **Step 1**: Boot from a trusted, clean medium (e.g., offline rescue disk) to scan or reinstall the OS, ensuring the rootkit's hooks are wiped.
    - **Step 2**: Patch the exploited vulnerabilities and enable secure boot/code-signing policies, so future unauthorized kernel modifications are blocked or detected.

C3. Diffie-Hellman key exchange without authentication

- **Attack**: An attacker can intercept and replace each party's public Diffie-Hellman share, creating two separate secret keys (one with each victim). The attacker relays messages, acting as a transparent proxy (MITM). Both victims think they communicate with each other.
- **Countermeasure**: Digitally sign each ephemeral public key or use certificates to verify that each side's key is genuine. This retains forward secrecy (ephemeral key) but ensures authenticity, defeating man-in-the-middle attacks.

End of Solutions